

Register-Spill Attribution

Memory gap in Table 4.4

Reduced Precision Hardware Acceleration of Nested Multi-Level Monte Carlo using CPUs

Mohammad Saifullah

September 2026

Table 4.4 reports that the FP32 level-0 basket kernel issues $5.20\times$ the L1 loads and $8.96\times$ the L1 stores of the FP16 kernel, against only $1.94\times$ on FP-arithmetic instructions.

The companion `docs/hardware_profiling_manual.tex` records how Table 4.4 itself was produced. Source and raw output are in <https://github.com/Saif2024C/mlmc-reduced-precision>; paths are relative to its root.

1 The hypothesis

The FP32 kernel obtains its Normals from Intel’s SVML routine `_mm512_cdfnorminv_ps`; the FP16 kernel uses Giles’ piecewise-linear superdyadic spline `norminv_fp16` (`src/AVX-512/my_mm512.h`). SVML is a library routine and is not inlined: it is reached by an indirect call. To cross into a routine it cannot see inside, the compiler must write every live vector register out to the stack, read them back on return, and rebuild any loop constant the routine has overwritten. The spline inlines completely and stays in registers.

The assembly code bears this out. Building `kernel_bench_basket_10.cpp` with `-g` and disassembling, the call in `kernel_fp32` is indirect through `%rbx`, which is loaded once before the loop with a fixed address:

```
4028bf: mov $0x405450,%rbx
```

and `nm` names that address:

```
0000000000405450 T __svml_cdfnorminvf8_19
```

The `f8` takes eight floats at a time, so a 16-lane iteration needs two calls. The loop body around them, verbatim:

```
402d2c: vpsubd %zmm18,%zmm21,%zmm21
402d32: vmovdqu64 %zmm21,0x540(%rsp)
402d3a: vptestnmd %zmm21,%zmm21,%k0
402d40: vpmovm2d %k0,%zmm0
402d46: vpsubd %zmm0,%zmm7,%zmm7
402d4c: vmovdqu64 %zmm7,0x580(%rsp)
402d54: vpsrld $0x9,%zmm1,%zmm0
402d5b: vcvt dq2ps %zmm0,%zmm0
402d61: vfmadd213ps %zmm20,%zmm19,%zmm0
402d67: vmovups %zmm0,0x4c0(%rsp)
402d6f: call *%rbx
402d71: vmovups %zmm0,0x500(%rsp)
```

```

402d79: vmovups 0x4c0(%rsp),%zmm0
402d81: vextractf64x4 $0x1,%zmm0,%ymm0
402d88: call *%rbx
402d8a: vmovups 0x540(%rsp),%zmm21
402d92: vmovups 0x580(%rsp),%zmm7
402d9a: vbroadcastss 0x112ac(%rip),%zmm20 # 414050
402da4: vbroadcastss 0x1129e(%rip),%zmm19 # 41404c
402dae: vpternlogd $0xff,%zmm18,%zmm18,%zmm18
402db5: vmovdqu64 0x11541(%rip),%zmm17 # 414300

```

Two registers go to the stack at 402d32 and 402d4c and come back at 402d8a and 402d92, either side of the calls. The argument is stored at 402d67 and reloaded at 402d79 to get at its upper half. From 402d9a onward the loop constants are rebuilt, the callee having overwritten them.

`kernel_fp16.kahan` contains no calls at all.

All of this recurs on every iteration. The listing shows where the traffic comes from; the measurement below shows how much of it there is.

2 The measurement

The two transforms are called on their own, with no path arithmetic and no payoff, so that whatever the RNG contributes is seen by itself. Each draws uniforms, converts them to Normals and accumulates the result so that nothing is optimised away. Both return sixteen Normals per iteration from one Philox draw, so the generator costs the same on each side and only the conversion differs.

Counters are read from inside the measured loop through `perf_event_open` rather than by wrapping the process in `perf stat`. Process startup, the seeding and the reporting are thus excluded, which matters because the loop is short. The four events are `instructions`, `cycles`, `L1-dcache-loads` and `L1-dcache-stores`, opened as one group so that no multiplexing occurs. The loop is run five times and the minimum of each counter taken.

$N = 4\,194\,304$ paths, matching Table 4.4.

3 Results

Per path, best of five, measured on `mimic`:

	loads	stores	insns	cycles
SVML <code>cdfnorminv</code>	5.44	1.94	18.81	12.25
spline <code>norminv_fp16</code>	0.19	0.06	7.31	10.65
ratio	29.00×	31.00×	2.57×	

Raw output is in `outputs/avx512/kernel/spill_attribution.txt`.

4 Reproducing

```

source /opt/intel/oneapi/setvars.sh
cd src/AVX-512/performance_analysis
icpx -O3 -march=native -std=c++17 \
    --gcc-install-dir=/usr/lib/gcc/x86_64-linux-gnu/13 \
    spill_test.cpp -o spill_test

```

```
./spill_test
```

The program reads the counters itself, so `perf` need not be installed and no root access is required. The one requirement is that the machine allows unprivileged users to read hardware counters, which on Linux means `kernel.perf_event_paranoid` set to 2 or lower; `mimic` already allows this.

To see the SVML calls in the disassembly:

```
icpx -O3 -march=native -std=c++17 -g \  
  --gcc-install-dir=/usr/lib/gcc/x86_64-linux-gnu/13 \  
  kernel_bench_basket_10.cpp -o kb_g \  
objdump -d --no-show-raw-insn kb_g | \  
sed -n '/<_ZL11kernel_fp32x>:/,/~/p' | grep -c call
```

This returns two calls per loop iteration for the FP32 kernel and none for `_ZL17kernel_fp16_kahanx`.

5 Limitations

This measures the transforms in isolation, not inside the estimator. It shows that SVML is expensive enough to explain Table 4.4's memory gap, but does not by itself establish that it does explain it; in the full kernel the cost is shared with genuine path work.

The test cannot say how much of SVML's cost is the call itself and how much is the work inside it. Everything here is single-threaded.